

Particle swarm algorithm with hybrid mutation strategy

Hao Gao^{a,b,*}, Wenbo Xu^b

^a Automation Department, Tsinghua University, Beijing 100084, China

^b School of Information Technology, Jiangnan University, Wuxi 214122, China

ARTICLE INFO

Article history:

Received 7 October 2008

Received in revised form 25 January 2011

Accepted 30 May 2011

Available online 12 June 2011

Keywords:

Particle swarm optimization

Monte Carlo Simulation

Henon map

Mutation

Power system

ABSTRACT

A new particle swarm optimization (PSO) that incorporates a hybrid mutation strategy is proposed. In this paper we first use the Monte Carlo method to investigate the behavior of the particle in PSO. The results reveal the essence of the particle's trajectory during executions and the reasons why PSO has relative poor global searching ability especially in the last stage of evolution. Then we present a new hybrid particle swarm optimization which incorporates Henon map mutation operation (HPSO) so as to enhance the achievement of PSO. The new mutation strategy divides the mutation operator into global and local mutation operators, then it enables the particles to have stronger exploration ability and fast convergence rate. Sixteen benchmark functions are used to test the performance of HPSO. The results show that the new PSO algorithm performs better than the other hybrid PSO algorithms for each of the test functions. Meanwhile, HPSO is applied to a practical problem (i.e., the economic dispatch problem in a power system) with a satisfying result.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

Particle swarm optimization (PSO) is a population-based stochastic optimization algorithm, firstly introduced by Kennedy and Eberhart [1]. It is a metaphor of the social behavior of animals, such as bird flocking and fish schooling. Comparing with other population-based stochastic optimization methods, such as Genetic Algorithms (GA) and Evolutionary Programming (EP), PSO has comparable or even superior search performance for many hard optimization problems with fast and stable convergence rate [2]. It has already been applied with success to many areas, such as function optimization [3–5], artificial neural network training [6,7], pattern classification and fuzzy system control [8,9]. However, PSO exhibits some disadvantages: it sometimes is easy to be trapped in local optima (it is often called premature convergence), and the convergence rate decreased considerably in the later period of evolution. A lot of existing methods attempt to improve the performance of PSO by introducing variable parameters [10,11], or making changes in the updating equation [12,13], or by adopting the operators of other optimization algorithms [14,15].

Mutation operators are an integral part of evolutionary computation techniques, preventing loss of diversity in a population of solutions, which allows a greater region of the search space to be covered. Furthermore, mutation operators introduce new individ-

uals into a population by creating a variation of a current individual, thus adding variability into the population and preventing stagnation of the search in local optima [16]. Therefore, the addition of mutation operators to PSO should enhance its global search ability and thus improve its performance. Recently, different hybrid PSOs with mutation operator have been proposed to overcome the drawback of PSO mentioned above. PSO with Gaussian mutation (GPSO) [17,18], whose global searching ability is more efficient than the standard PSO, is proposed by Natsuki and Andrews. Wang proposed the PSO with Cauchy mutation (HCP SO, IPSO) [19,20], by adding a Cauchy mutation on the *pbest* or the *gbest* particle, the mutated best particle could lead all the rest of particles to the better positions. Ling et al. [21] proposed a hybrid PSO with wavelet mutation named HWPSO, in which a fine-tuning mutating method is used. In HWPSO, by performing mutation operation on the selected particle, the solution space should be explored more efficiently and premature convergence is more likely to be avoided. Chaos mutation strategy also has been used to improve the PSO's searching ability. Coelho [22] introduces chaotic mapping into PSO for improving the global searching ability (CGPSO). Although different mutation operators have been applied to enhance the achievements of PSO, the new mutation strategy needs to be further investigated so as to achieve better balance between exploration and exploitation searching abilities.

In this paper, we first analyze the mechanism of PSO. There are two random-variables in PSO and random property is the essential characteristic of PSO [2]. But almost all the past mathematical analyses on the mechanism of PSO are based on assuming these random-variables as constant [23,24], they do not provide

* Corresponding author at: Dept. Automation Room 725, Central Main Building, Tsinghua University, China. Tel.: +86 10 6278 8613 832.

E-mail address: gaohao.wuxi@yahoo.com (H. Gao).

an effective mathematical tool for analyzing the influence of these random-variables to the achievement of PSO. In this paper we introduce the Monte Carlo method to analyze the mechanism of PSO by considering the random property. Second, a new mutation strategy incorporating the Henon map is proposed in this paper. The hybrid PSO algorithm first uses the global henon mutation operator to enhance the global searching ability of particles. Then the local henon mutation operator is employed to improve the convergence speed. As the results of experiments show, HPSO gets an effective balance between the global and local searching abilities. Finally, an industrial application example on power system problem is used to illustrate the performance and the applicability of the proposed hybrid PSO.

The rest of this paper is organized as follows. Section 2 introduces the standard PSO shortly and Monte Carlo Simulation is then used to show the global searching ability of PSO in Section 3. The proposed PSO, which has global and local mutation operators, is presented in Section 4. Computational experiences with a number of test functions from the literature are shown in Section 5. The parameters of HPSO are also analyzed in this section. Section 6 presents the experimental studies on the problem of Economic Dispatch for power system. Finally, Section 7 summarizes and draws conclusions.

2. Particle swarm optimization

PSO starts with the random initialization of a swarm of particles in the search space and works on the social behavior of the particles in the swarm. In PSO algorithm, each member of the population is called a “particle”, and each particle “flies” around in the n -dimensional search space with a velocity, which is constantly updated by the particle's own experience and the experience of the whole swarm. The experience of a particle is recorded from the previous iteration and represented as $pbest$; the experience among all the particles is represented as $gbest$. Each particle in PSO has a position and a velocity, its evaluation is achieved using the objective function of the optimization problem, whose variables are the particle position dimensions. The particle updating method tries to move particles to better positions by accelerating them towards $pbest$ and $gbest$. The basic PSO algorithm can be described as follows:

$$v_{id}(t+1) = wv_{id}(t) + c_1 rand_1 * (pbest_{id} - x_{id}(t)) + c_2 rand_2 * (gbest_d - x_{id}(t)) \quad (1)$$

$$x_{id}(t+1) = x_{id}(t) + v_{id}(t+1) \quad (2)$$

where c_1 and c_2 are positive constants and called the acceleration coefficients, $rand_1$ and $rand_2$ are two random functions in the interval $[0,1]$. v_{id} is the velocity of individual i on dimension d . x_{id} is i 's current position on dimension d . $pbest_{id}$ is the location of the best problem solution vector found by i ; and $gbest$ represents the best solution found so far by all particles in the population. In Eq. (1), w is the inertia weight which provides the necessary diversity to the swarm by changing the momentum of particles. Generally, a maximum velocity (V_{max}) for each modulus of the velocity vector of the particles is defined in order to control excessive roaming of particles outside the user defined search space. Whenever a v_{id} exceeds the defined limit, its velocity is set to V_{max} .

3. Analysis of the particle's behavior

A series of mathematical analyses on the mechanism of PSO has been done on the analysis and development of PSO since it was introduced in 1995. In [23] Clerc presented each particle in the PSO

system converges to its one and only local attractor of point $Q = (Q_1, Q_2, \dots, Q_n)$, of which the coordinates are:

$$Q_d = \frac{c_1 rand_1 * pbest_{id} + c_2 rand_2 * gbest_d}{c_1 rand_1 + c_2 rand_2} \quad (3)$$

Van den Bergh performed a movement analysis of one particle in detail and used Gaussian-elimination to calculate the recurrence relation [24], then concluded that the trajectory of particle can be guaranteed to converge if the parameters c_1 , c_2 and w are chosen according to formula (4):

$$w > \frac{1}{2} * (c_1 + c_2) - 1 \quad (4)$$

These studies mentioned above are based on a kind of reduced system that results from (1) and (2) by simplifying random operators $\Phi_1 = c_1 rand_1$, $\Phi_2 = c_2 rand_2$ as constants. In fact nobody knows whether the individual-learning terms or social-learning terms should be stronger during iterations and the two random variables ($rand_1$, $rand_2$) in Eq. (1) essentially reflect the random property of PSO [2]. So we must analyze the influence of these random parameters on the search ability of particles.

According to Eq. (2), the change of particle's position only depends on the velocity-item, so we only use the velocity-item to investigate the search ability of particles during executions. The implicit form of the velocity updating equation, presented in Eq. (1), is used for multi-particle PSO working in a multi-dimensional search space. Due to the fact that the data of each dimension in PSO algorithm is independent, the analysis below will be restricted to a single dimension so that the subscript j is dropped. The notation can be simplified even more by looking at the trajectory of a single particle in isolation, so we drop the subscript i . Now the following equations are obtained by changing the format of (1) and (2):

$$V(t+1) = wV(t) + \Phi_1(pbest - X(t)) + \Phi_2(gbest - X(t)) \quad (5)$$

$$X(t+1) = V(t+1) + X(t) \quad (6)$$

Then Eqs. (5) and (6) also can be written as:

$$-X(t-1) = \frac{1}{\Phi_3 + \Phi_4} * (V(t) - wV(t-1) - \Phi_3 pbest - \Phi_4 gbest) \quad (7)$$

$$X(t) = V(t) + X(t-1) \quad (8)$$

where $\Phi_i = c * rand_i$, ($c = c_1 = c_2$, $i = 1, 2, 3, 4$). Then by substituting Eqs. (8), (7) into Eq. (5), the following non-homogeneous recurrence relation is obtained:

$$V(t+1) = (w - s + q) * V(t) - wq * V(t-1) - \mu * (pbest - gbest) \quad (9)$$

where $s = \Phi_1 + \Phi_2$, $q = (\Phi_1 + \Phi_2) / (\Phi_3 + \Phi_4)$, $\mu = (\Phi_1 \Phi_4 - \Phi_3 \Phi_2) / (\Phi_3 + \Phi_4)$.

The first and second items, on the right side of Eq. (9), memorize the past value of velocity. So they can be seen as a whole item named as the memory-item of velocity. Since the value of the third item on the right side of Eq. (9) obtains from the past experience of particles, we call it as learning-item of velocity [11]. According to Eq. (9), q in memory-item and μ in learning-item are both four-dimensional random-variables, thus it is very inconvenient to directly calculate their change during evolution. As Boyle described in [25], for problems which are so complex for calculation, Monte Carlo method is an effective method to get precise results by a large number of relative experiments. So we introduce the Monte Carlo method to analyze the property of memory-item and learning-item separately by consideration of a large number of experimental data.

Since the memory-item of Eq. (9) is a recurrence relation, according to the Monte Carlo theory, we set the experimental

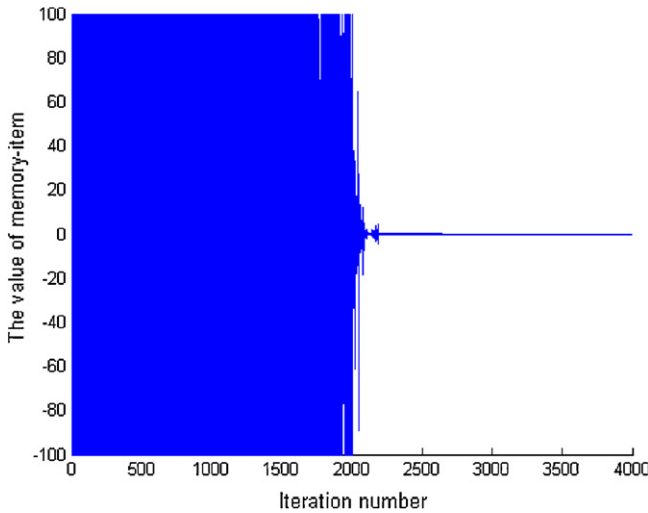


Fig. 1. The memory-item's change of 1000 particles during 4000 iterations using Monte Carlo method.

number of particle, $V(0)$, $V(1)$, $V_{\max}$ and maximum number of iterations as 1000, $\text{rand}(100)$, $\text{rand}(100)$, 100 and 4000. For showing the power of the learning-item, we calculate the probability density distribution of coefficient μ . The number of experiments is set as 50,000. c is set as 2; w is set as 0.9, and linearly decreasing to 0.4 at the end of evolutionary processing, which guarantees the convergence behavior of PSO algorithm [26].

In our test, we first investigate the power of the memory-item. According to Fig. 1, under the condition of $pbest = gbest$, all the memory-item of 1000 particles decrease to almost zero after about 2200 iterations. Then we can infer that, only by using the memory-item, particles with large initial velocity have no chance to escape from their local search space after the limited iterations.

Now, let us investigate the impact upon the particle of the power of the learning-item. From Fig. 2, according to the symmetry property, μ belongs to $(-2, 2)$ and the mean value of μ is 0. Referring to the convergence property of PSO, if the particle's $pbest$ is close to $gbest$, the power of the learning-item is too small to help the particle to jump out of its local area.

In general, the memory-item of velocity only helps the particle roam through the searching space in the limited iterations. Furthermore, once the particle's $pbest$ is close to $gbest$, the learning-item is helpless to enhance the global searching ability of particle. Then we

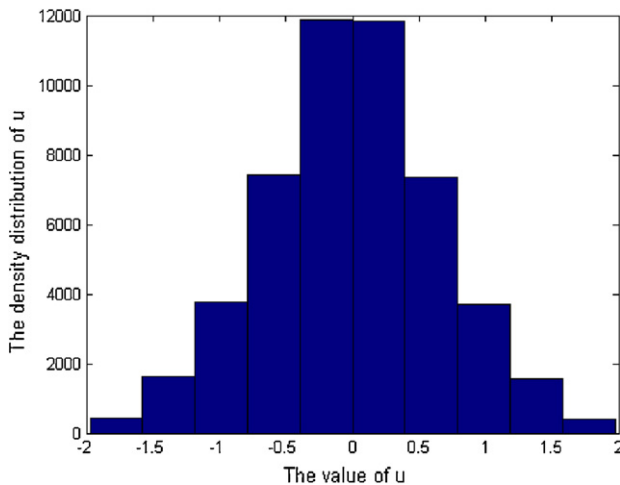


Fig. 2. The probability density distribution histogram of μ using Monte Carlo method.

can conclude that at the beginning of evolution, since the swarm's velocity is bigger and the diversity of particles is higher, PSO has relatively strong global search ability. However, in the last stage of evolution, PSO algorithm has no chance to escape from the local area of $gbest$.

4. Hybrid particle swarm optimization

4.1. Hybrid PSO algorithm

For Evolutionary Algorithms, such as GA and EP, the solution space is more likely to be explored by setting a larger mutating operator, and it is more likely to be fine-tuned for a better solution by setting a smaller mutating space [28]. This idea can be applied to enhance the achievement of PSO. Here we propose a new mutation strategy which not only effectively enhances the global searching ability of PSO but also improves the fast convergence rate of PSO. The new algorithm we call hybrid PSO (HPSO).

In HPSO, we set a velocity-threshold denoted as T_d to measure whether the current particle can use the mutation operator on dimension d at iteration t . A big value of T_d should lead to the chaos of particles. And a small one results in the particle needing a relatively long time to use the mutation operator, then HPSO needs more time to escape from local optima. In view of above, we use gradient descent method to set T_d . When the number of particles which have used the mutation operator, denoted as F_d , equals the specified value k , T_d should be updated by using Eq. (10).

$$\text{if } (F_d = k) \text{ then } F_d = 0; \quad T_d = \frac{T_d}{m} \quad (10)$$

Parameter k controls the mutation frequency of particle and m controls the decline rate of T_d . They both influence the achievement of HPSO. Simulations are carried out to find out the best choices of m and k in Section 5, Part 3. F_d is updated by using Eqs. (10) and (11).

$$F_d = F_d + \sum_{i=1}^S b_{id}(t) \quad (11)$$

and

$$b_{id}(t) = \begin{cases} 0, & \text{if } \text{abs}(x_{id}(t) - x_{id}(t-1)) > T_d \\ 1, & \text{if } \text{abs}(x_{id}(t) - x_{id}(t-1)) \leq T_d \end{cases}$$

where T_d , $F_d(0)$ are set to $x_{\max}$ and 0 at the beginning, and S is the number of particles in population.

Once the absolute value of $v_{id}(t)$ has decreased to T_d , HPSO should use the global or local mutation operator. The global operator is used to enhance the global search ability and the local operator allows the particles to make a precise search in the local area. The global mutation operator is described as follows:

$$\text{mutate}(v_{id}(t+1)) = (\text{henon}(k1) * V_{\max}); \quad pbest_{id} = x_{id}(t) + v_{id}(t+1) \quad (12)$$

$\text{henon}(k1)$ returns a value generated by Henon map. In Section 4, Part 2, it is shown that Henon map can give particles more opportunity to jump out of the local area. $k1$ represents the number of times in which HPSO has used the global mutation operator. So as to allow the mutated particle to have more chance to search in the global area, $pbest_{id}$ is also updated.

By using the global mutation operator, HPSO has more opportunity to search in the global solution space, but a large step size may not be beneficial at all if the current search point is already very close to the global optimum. So in this paper we introduce a

```

if  $f(pbest_i) < f(gbest)$ 

     $gbest = pbest_i$  and  $f(gbest) = f(pbest_i)$ 

end if

end if

if  $(F_d > k/2)$ 

    mutate  $gbest$  using Eq. (13)

    if  $f(gbest') < f(gbest)$ 

         $gbest = gbest'$  and  $f(gbest) = f(gbest')$ 

    end if

end if

end if

update  $F_d$  and  $T_d$  using Eq. (10), (11)

end for

update the particle position  $x_i(t+1)$  using Eq. (2)

evaluate  $f(x_i(t+1))$ 

end for

end do

end

```

Fig. 3. Pseudo code for the HPSO algorithm.

local mutation operator into the hybrid PSO, then the mutated particle has more opportunity to make a precise searching in the local space. The local mutation operator is described as follows:

$$gbest' = gbest * (1 \pm henon(k2)) \quad (13)$$

$henon(k2)$ returns a number generated by Henon map. In Section 4, Part 2, we prove that Henon map is fit for use as the local mutation operator in HPSO. $k2$ represents the number of times that HPSO has used the local mutation operator.

As shown in Eq. (11), if T_d is a relative small value especially in the last stage of evolution, then the particle in HPSO should take a relatively long time before using mutation operators. For further enhancing the searching ability of HPSO, whenever the following equation is satisfied:

$$\left(rand < \left(\frac{1}{S * n} \right) \right) \quad (14)$$

where n represents the number of dimension in the search space. HPSO also uses the mutation operators.

The main steps of HPSO algorithm are given in Fig. 3.

4.2. Mutation operator

In the past years, many mutation operators have been applied to PSO. Hiquishi and Iba [17] and Andrews [18] used a Gaussian mutation operator that changes a particle position value. Wang and Liu

[19] introduced a mutation operator similar to that of Hiquishi and Iba [17], but used a Cauchy probability distribution. Ling et al. [21] proposed a hybrid PSO with Morlet wavelet mutation. The Gaussian and Cauchy density functions centered at the origin are defined by:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right), \quad -\infty < x < \infty$$

$$f(x) = \frac{1}{\pi} * \frac{t}{t^2 + x^2}, \quad -\infty < x < \infty$$

The Morlet wavelet [21] is presented as:

$$\psi(x) = e^{-x^2/2} \cos(5x)$$

The Morlet Wavelet integrates to zero and over 99% of the total energy of the function is contained in the interval of $-2.5 \leq x \leq 2.5$.

The Henon map used in [22] is given by:

$$y_1(k) = 1 - 1.4(y_1(k-1))^2 + y_2(k-1), \quad y_2(k) = 0.3y_1(k-1)$$

$$henon(k) = \frac{y_1(k) + 1.2846}{1.2846 + 1.2719}$$

with $y_1(0) = 0.63135448$; $y_2(0) = 0.15940633$.

In this paper, using the Monte Carlo method, we investigate which function is fit for finding the global optimum and which has the best fine-tuning searching ability. The experimental number, σ in Gaussian function, t in Cauchy function, x in Morlet wavelet are set as 10000, 0.16, 0.32, 2.5. As described in Eqs. (12) and (13), the formulas of global and local mutations are different. So we investigate the role of them separately.

From Fig. 4, it is clear that if the Henon function is used as the global mutation operator, it is more likely to generate an offspring further away from its parent among the considered functions since almost 70% data of Henon map shown in Fig. 4(d) is larger than 0.5. It is expected to have a higher probability of escaping from a local optimum, especially when the plateau is large relative to the mean step size.

As it is shown in Fig. 5, the Henon map also shows the smallest hill of the center. It indicates that Henon mutation has more opportunity to exploit the local neighbourhood of $gbest$ and thus has a weaker fine-tuning ability than the others in small to mid-range regions. In the Section 5, Part 3, the empirical results support the above intuitions.

5. Performance evaluation

5.1. Benchmark test functions

To test the performance of HPSO, sixteen benchmark functions [21,29] listed in Table 1 are used here for comparison with standard PSO [27], CGPSO [22], HWPSO [21], HCPSPSO [19], IPSO [20], GPSO [18] and HPSO algorithms. The range of population initialization, the dynamic range of each function and the global optimum of these functions are shown in Table 1. Work done by van den Bergh and Engelbrecht [30] showed that the effect of population size on the performance of PSO method is of minimum significance. So in this paper all empirical experiments are carried out with a population size of 20. Furthermore, in order to ensure the initial values of particles in each algorithm are the same, we use the MATLAB command `rand('state', sum(i*30))`. The value of i used at the beginning of each run in all algorithms is the same. The following parameters of HPSO are used:

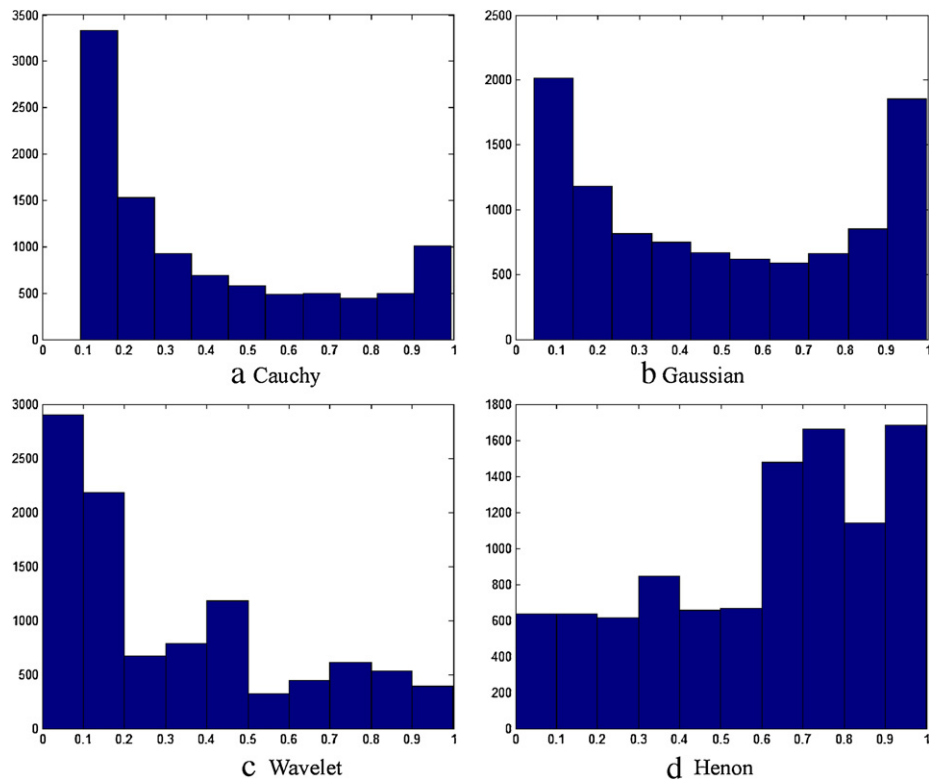


Fig. 4. Comparison between Gaussian, Cauchy, Morlet wave and Henon functions for global mutation operator.

- $w = 0.7298$, $c_1 = c_2 = 1.49618$.
- Number of runs: 50.
- Swarm size: 20.
- The parameter k of HPSO: 10.
- The parameter m of HPSO: 10.

To show the merits of our algorithm, four categories of experiment are carried out in our test. In unimodal functions (Category 1), multimodal functions (Category 2) and composite functions (Category 4) tests, the maximum number of iterations are set as 2000, 4000, 6000 and 8000 for solving 10-D, 20-D, 30-D and 40-D prob-

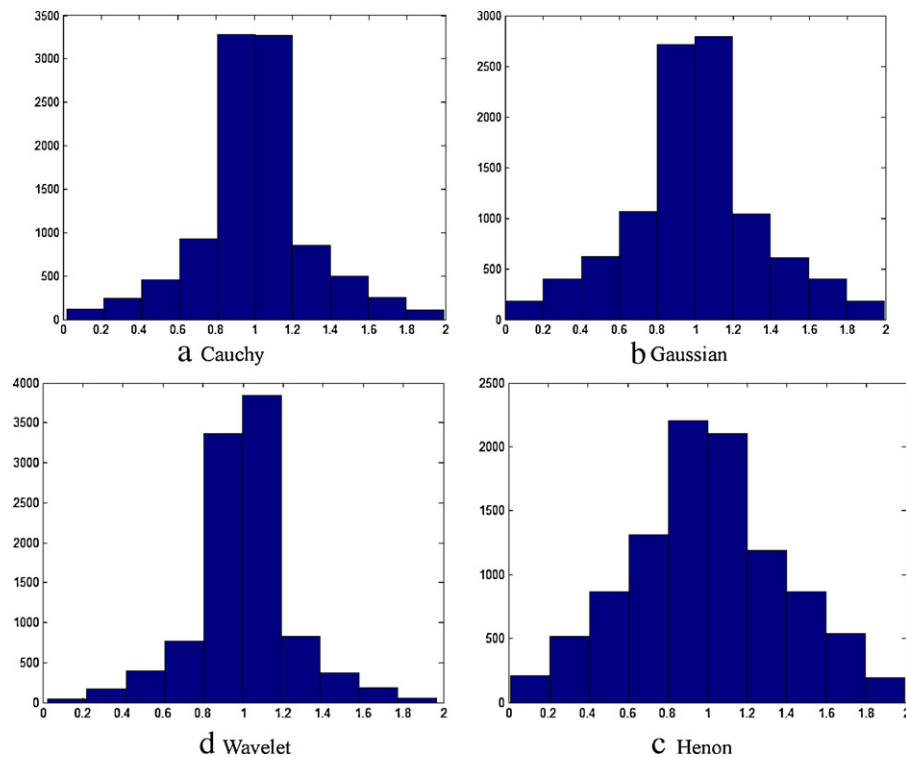


Fig. 5. Comparison between Gaussian, Cauchy, Morlet wave and Henon functions for local mutation operator.

Table 1
Benchmark test functions.

Function	Formula	Range	$V_{\max}$	$f_{\min}$
f_1	$\sum_{i=1}^n x_i^2$	$[-100,100]$	100	0
f_2	$\sum_{i=1}^n i * x_i^2$	$[-100,100]$	100	0
f_3	$\sum_{i=1}^n ([x_i] + 0.5)^2$	$[-100,100]$	100	0
f_4	$\sum_{i=1}^n x_i + \prod_{i=1}^n x_i $	$[-100,100]$	100	0
f_4	$\sum_{i=1}^n i x_i^4 + \text{random}[0, 1]$	$[-1.28, 1.28]$	2.56	0
f_6	$\sum_{i=1}^n \max x_i $	$[-100,100]$	100	0
f_7	$\sum_{i=1}^n (x_i^2 - 10 \cos(2\pi x_i) + 10)$	$[-5.12, 5.12]$	10	0
f_8	$-20 \exp \left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left(\frac{1}{n} \sum_{i=1}^n \cos 2\pi x_i \right) + 20 + e$	$[-32, 32]$	32	0
f_9	$\frac{\pi}{n} \left\{ 10 \sin^2(\pi y_1) + \sum_{i=1}^{n-1} (y_i - 1)^2 * [1 + 10 \sin^2(\pi y_{i+1})] + (y_n - 1)^2 \right\}$	$[-50, 50]$	50	0
f_{10}	$0.1 \left\{ \sin^2(3\pi x_1) + \sum_{i=1}^{n-1} (x_i - 1)^2 * [1 + \sin^2(\pi x_{i+1})] + (x_n - 1)^2 [1 + \sin^2(2\pi x_n)] \right\}$	$[-50, 50]$	50	0
f_{11}	$- \left[2.5 \prod_{i=1}^n \sin \left(x_i - \frac{\pi}{6} \right) + \prod_{i=1}^n \sin \left(5 \left(x_i - \frac{\pi}{6} \right) \right) \right]$	$[0, 180]$	180	-3.5
f_{12}	$\sum_{i=1}^n (x_i - 1)^2 - \sum_{i=2}^n x_i x_{i-1}$	$[-n^2, n^2]$	n^2	$f(x^*)$
f_{13}	$0.5 + \frac{(\sin \sqrt{x_1^2 + x_2^2})^2 - 0.5}{(1 + 0.001(x_1^2 + x_2^2))^2}$	$[-100, 100]$	100	0
f_{14}	$(x_1^2 + x_2^2)^{0.25} (\sin^2(50(x_1^2 + x_2^2)^{0.1}) + 1)$	$[-100, 100]$	100	0
f_{15}	$(\exp(x_1) - x_2)^4 + 100(x_2 - x_3)^6 + (\tan(x_3 - x_4))^4 + x_4^8$	$[-1, 1]$	1	0
f_{16}	$2x_1^2 - 1.05x_1^4 + \frac{1}{6}x_1^6 + x_1x_2 + x_2^2$	$[-5, 5]$	5	0

lems except the maximum number of iterations for f_3 is set as 200, 400, 600 and 800. In low-dimensional functions (Category 3), the maximum number of iteration is set as 1000. In the test of unimodal functions, we not only give the final achievements of each algorithm but also show the convergence rate of some algorithms. For multimodal and low dimensional functions, the final results are much more important since they reflect an algorithm's ability of escaping from poor local optima and locating a good near-global optimum. The experimental results of HPSO in terms of the average final best value and the standard deviation over 50 runs are summarized in Tables 2–5. The best result in a comparison is emphasized as bold font.

5.2. The experimental results of HPSO

5.2.1. Category 1 – unimodal functions

Table 2 shows the performance of HPSO on unimodal functions. Among the compared algorithms, only HPSO gets the global optima on each unimodal test function except f_5 . Function f_5 is Schwefel's problem 2.21 and HPSO also gets the best and stable achievements on this function.

Compared with the mutation operator used in the other hybrid PSOs, the role of the local mutation operator used in HPSO is only to make precise searching nearby g_{best} . It does not bear the burden of roaming through the searching space. This means that particles in HPSO have more chance to search in the local area of g_{best} . The rapid

convergence of HPSO shown in Fig. 6 also supports our explanations. In short, HPSO is the best of the methods compared to tackle unimodal functions.

5.2.2. Category 2 – multimodal functions

Function $f_7 - f_{12}$ are such functions where the number of local minima increases exponentially as the dimension of the function increases. In the comparison of this category, we mainly investigate the global searching ability especially in each of the hybrid PSOs with mutation operator.

Function f_7 is the generalized Rastrigin's function. It is the most commonly used test function in PSO algorithm. An optimization algorithm can easily become trapped in a local minimum on its way to the global minimum. Depending on having better balance between the local and global searching ability, it can be clearly seen from Table 3 that the mean value of HPSO gets the global optima on each dimension. Thus the HPSO can provide higher-quality results on f_7 . Function f_8 is Ackley function, the difference of each compared algorithm is not obvious on dimension 10 and 20, but with the dimension increasing to 30 and 40, the mean value and the standard deviation of the HPSO are obviously more precise and stable than those of the other algorithms. Function f_9 and f_{10} are Levy and Montalvo functions which have approximately 5^n and 15^n local minima. From Table 3 HPSO shows the best results on both functions. Moreover, when the dimension of f_{10} is increased to 20, 30 and 40, all the compared algorithms except HPSO are trapped in

Table 2
Comparison between different PSO methods for benchmark test functions (Category 1).

Function	Dimension	Average (Std. dev.)						
		PSO	CGPSO	HWPSO	HCPSO	IPSO	GPSO	HPSO
f_1	10	2.1485e–43 (8.451e–43)	7.621e–85 (5.07e–84)	6.2861e–56 (1.506e–55)	5.2294e–110 (3.658e–109)	3.6188e–93 (1.215e–92)	2.1113e–43 (1.364e–42)	0 (0)
	20	7.949e–36 (3.78e–35)	2.647e–76 (9.03e–76)	6.272e–45 (2.03e–44)	2.1313e–102 (1.48e–101)	2.716e–99 (1.54e–98)	9.3791e–37 (2.82e–36)	0 (0)
	30	1.7e–29 (9.92e–29)	6.8236e–51 (2.74e–50)	3.79e–36 (1.356e–35)	2.904e–81 (1.069e–80)	4.338e–83 (1.543e–82)	2.488e–30 (9.81e–30)	0 (0)
	40	7.9543e–25 (3.41e–24)	7.6471e–31 (4.12e–30)	6.654e–30 (3.99e–29)	4.64e–61 (8.65e–46)	3.28e–63 (5.91e–40)	1.5675e–25 (4.78e–25)	0 (0)
f_2	10	1.9757e–43 (2.126e–42)	9.54e–84 (3.66e–83)	1.8459e–55 (7.78e–55)	4.1417e–113 (1.88e–112)	5.521e–92 (3.79e–91)	8.8925e–44 (4.28e–43)	0 (0)
	20	2.377e–34 (2.28e–34)	3.5926e–74 (1.43e–73)	5.4521e–44 (1.93e–43)	1.488e–103 (5.42e–103)	1.2738e–95 (8.86e–95)	3.6864e–36 (1.35e–35)	0 (0)
	30	4.0583e–28 (1.38e–27)	2.5488e–50 (1.36e–49)	6.354e–36 (1.91e–35)	9.268e–80 (6.342e–79)	6.23e–80 (4.239e–79)	5.969e–30 (1.356e–29)	0 (0)
	40	4.503e–24 (1.352e–23)	3.564e–31 (2.49e–30)	3.563e–29 (1.107e–28)	1.823e–58 (1.184e–57)	7.5156e–61 (3.52e–60)	3.8618e–24 (5.89e–24)	0 (0)
f_3	10	0.1333 (0.33993)	0.46667 (0.88443)	0.13333 (0.33993)	0.033333 (0.17951)	0.033333 (0.17951)	0.13333 (0.33993)	0 (0)
	20	1.8333 (1.5074)	24.633 (30.751)	2.9333 (2.7439)	0.76667 (1.3085)	1.6667 (2.0548)	2.2333 (2.3192)	0 (0)
	30	10 (6.5269)	938.27 (1805.2)	9.2333 (6.1455)	1.1667 (1.2671)	4.2333 (2.0766)	8.1333 (6.0096)	0 (0)
	40	26.4 (16.637)	3820.6 (3307.6)	33.867 (22.485)	2.6 (2.2891)	16.067 (9.377)	21.467 (10.181)	0 (0)
f_4	10	3.6751e–26 (6.54e–26)	6.8199e–32 (4.75e–31)	1.8378e–31 (9.69e–31)	2.8875e–38 (2.012e–37)	3.358e–46 (2.332e–45)	2.9261e–26 (7.097e–26)	0 (0)
	20	4.0631e–23 (7.226e–23)	1.1132e–10 (7.62e–10)	3.834e–22 (1.866e–21)	1.4707e–17 (7.16e–17)	5.2426e–19 (3.16e–18)	9.2341e–24 (2.76e–23)	0 (0)
	30	4.099e–20 (7.747e–20)	0.0718 (0.4344)	4.052e–17 (1.993e–16)	1.156e–5 (8.093e–5)	5.824e–7 (3.838e–6)	1.387e–20 (4.044e–20)	0 (0)
	40	7.1593e–18 (1.239e–17)	0.109 (0.6846)	6.369e–14 (2.915e–13)	1.2522e–7 (8.07e–7)	1.6581e–5 (1.093e–4)	1.9846e–16 (1.037e–15)	0 (0)
f_5	10	0.0031 (0.0014)	0.0067527 (0.007793)	0.0015985 (0.001482)	0.0024657 (0.002089)	0.002574 (0.002997)	0.0016308 (0.001791)	0.00002706 (0.0000253)
	20	0.0088624 (0.003757)	0.03501 (0.044516)	0.0053655 (0.00238)	0.0088792 (0.005128)	0.0092155 (0.00086)	0.0049364 (0.0029977)	1.7693e–4 (1.2638e–4)
	30	0.0205 (0.0087124)	0.1109 (0.1306)	0.0127 (0.0042)	0.0299 (0.0437)	0.0218 (0.0161)	0.0094193 (0.0037513)	8.137e–5 (7.4733e–5)
	40	0.037924 (0.0138)	0.3078 (0.3253)	0.023 (0.0063)	0.0733 (0.1136)	0.0383 (0.028)	0.016 (0.0049)	6.3586e–5 (6.2862e–5)
f_6	10	6.2832e–10 (1.91e–9)	7.7112e–27 (9.42e–27)	8.757e–9 (6.223e–9)	1.1236e–30 (3.487e–30)	6.072e–30 (4.474e–29)	1.0564e–10 (2.438e–10)	0 (0)
	20	0.0303 (0.0295)	3.428e–8 (8.586e–8)	0.0205 (0.0167)	6.4296e–10 (1.9073e–9)	3.2482e–10 (7.4371e–10)	0.0139 (0.0128)	0 (0)
	30	2.8481 (1.0204)	0.013627 (0.013311)	0.6206 (0.2056)	0.0014 (0.0016)	0.0016 (0.0022)	1.3156 (0.4644)	0 (0)
	40	12.2652 (2.5873)	1.3132 (0.79308)	2.38 (0.5956)	0.1569 (0.1896)	0.153 (0.1335)	6.2652 (1.8406)	0 (0)

Table 3
Comparison between different PSO methods for benchmark test functions (Category 2).

Function	Dimension	Average (Std. Dev.)						
		PSO	CGPSO	HWPSO	HCPSO	IPSO	GPSO	HPSO
f_7	10	4.1788 (2.332)	12.47 (5.2317)	4.61 (2.5364)	4.9085 (2.8367)	4.7344 (2.2011)	4.1857 (2.2241)	0 (0)
	20	15.687 (4.9527)	47.824 (12.368)	19.667 (6.7661)	24.244 (7.8153)	19.775 (5.6355)	16.815 (6.745)	0 (0)
	30	39.168 (12.497)	88.153 (21.92)	44.723 (13.968)	51.273 (13.328)	46.365 (12.6)	34.956 (8.2597)	0 (0)
	40	61.323 (11.567)	137.04 (28.866)	66.928 (15.817)	81.752 (14.099)	86.563 (21.899)	56.149 (15.351)	0 (0)
f_8	10	2.783e–15 (6.3774e–15)	0.37928 (0.59006)	3.2567e–15 (4.324e–15)	0.038505 (0.20736)	2.9014e–15 (8.854e–16)	3.1382e–15 (4.2077e–15)	8.8817e–16 (3.94e–31)
	20	6.8094e–15 (2.5445e–15)	2.4989 (1.2489)	7.6383e–15 (2.8422e–15)	0.07701 (0.28814)	0.085956 (0.32348)	8.112e–15 (3.005e–15)	8.8817e–16 (3.94e–31)
	30	1.9718e–14 (8.0703e–14)	4.9135 (1.564)	1.9836e–14 (4.405e–14)	0.75882 (0.8446)	0.82488 (3.5277)	2.102e–14 (9.755e–15)	8.8817e–16 (3.94e–31)
	40	0.033078 (0.17813)	6.688 (1.9832)	0.13993 (0.42729)	1.5692 (0.85021)	1.4467 (3.3523)	2.8664e–13 (3.2871e–13)	8.8817e–16 (3.94e–31)
f_9	10	4.7116E–32 (3.2843E–32)	0.041468 (0.10572)	4.7116E–32 (3.284E–31)	4.7116E–32 (3.2843E–32)	0.010367 (0.055827)	4.7116E–32 (3.2843E–32)	4.7116E–32 (3.284E–32)
	20	0.015549 (0.061545)	8.0548 (23.573)	0.051833 (0.1291)	0.036283 (0.07705)	0.11418 (0.3243)	0.020734 (0.05286)	6.0304e–10 (7.95e–10)
	30	0.055315 (0.11903)	9.1704 (14.618)	0.06914 (0.2115)	0.27351 (0.7275)	0.07082 (0.11234)	0.06565 (0.0824)	3.742e–9 (1.387e–9)
	40	0.057115 (0.12883)	7.3681 (16.986)	0.63258 (3.0087)	3.0924 (16.177)	0.11678 (0.26122)	0.077843 (0.1672)	6.6299e–9 (5.308e–9)
f_{10}	10	1.3498e–32 (1.095e–47)	62.159 (102.45)	4.7197e–31 (5.8274e–32)	2.1309e–32 (4.2062e–32)	61.529 (83.1694)	1.3498e–32 (1.095e–47)	1.3498e–32 (1.095e–47)
	20	32.201 (52.647)	841.73 (229.76)	65.348 (73.355)	178.64 (172.81)	126.78 (101.34)	24.385 (41.532)	1.0115e–7 (1.5452e–7)
	30	298.16 (165.89)	1685.3 (351.37)	271.46 (190.71)	939.96 (367.96)	1023.7 (228.21)	311.04 (198.98)	7.7956e–7 (9.119e–7)
	40	850.98 (378.82)	2614.6 (364.94)	656.32 (331.19)	1762.6 (382.23)	1937.4 (321.94)	799.53 (295.78)	1.4074e–6 (1.208e–6)
f_{11}	10	–1.4854 (0.8135)	–1.5181 (0.2491)	–3.16388 (0.6077)	–2.7087 (0.4752)	–2.7604 (0.99993)	–1.944 (0.803557)	–3.4538 (0.10061)
	20	–0.24197 (0.12493)	–0.32467 (0.14492)	–2.2421 (1.136)	–1.1394 (0.61723)	–1.3902 (0.89227)	–0.2896 (0.269965)	–3.2328 (0.6402)
	30	–0.087188 (0.020551)	–0.029155 (0.022926)	–1.5484 (1.02933)	–0.94511 (0.69244)	–1.16005 (0.57064)	–0.06554 (0.03837)	–3.0415 (0.917)
	40	–0.033027 (0.0084484)	–0.0059837 (0.005442)	–1.0442 (0.70762)	–0.74892 (0.5247)	–1.23585 (0.77528)	–0.01678 (0.01777)	–3.2311 (0.6729)
f_{12}	10	–209.99992 (0.0019)	–210 (0.447e–6)	–209.9982 (0.0029)	–209.9998 (5.282e–4)	–210 (7.66e–6)	–209.999 (0.0022)	–210 (5.482e–11)
	20	–1264.942 (186.957)	–1519.5 (0.72174)	–1366.486 (132.656)	–1510.297 (13.318)	–1519.98 (0.009934)	–1291.4708 (172.5639)	–1520 (0.0043)
	30	–2403.384 (572.65)	–4714.3 (244.36)	–2667.99 (652.41)	–4847.1856 (59.545)	–4851.889 (19.844)	–2253.9811 (547.959)	–4858.8795 (13.2867)
	40	–3069.924 (944.128)	–8306.3 (2776.8)	–3372.6825 (955.6055)	–10150.309 (286.3964)	–9873.086 (165.5756)	–3124.983 (1106.433)	–10238.2 (321.09)

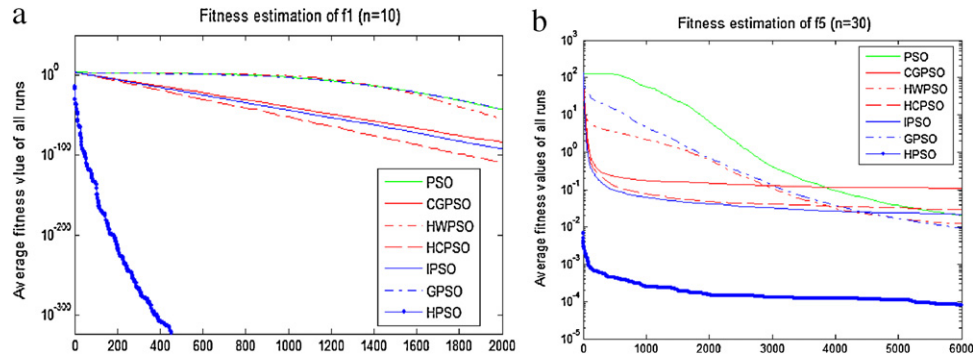


Fig. 6. (a) Function 1 in 10 dimensions. (b) Function 5 in 30 dimensions.

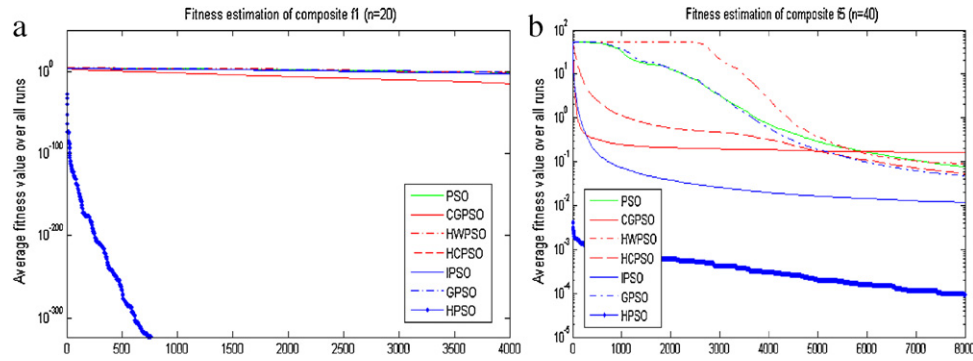


Fig. 7. (a) Composite Function 1 in 20 dimensions. (b) Composite Function 5 in 40 dimensions.

the local minima. This is mainly attributed to the global searching ability which enables HPSO to have more opportunities to search in the global area. Function f_{11} is Sinusoidal Problem. The location of the global solution is as $x_i^* = 90 + \pi/6$ with the number of local minima equal to

$$\sum_{i=0}^{\lfloor n/2 \rfloor} \left(\frac{n!}{(n-2i)!(2i)!} 3^{n-2i} 2^{2i} \right)$$

According to the results shown in Table 3, in the compared algorithms only HPSO gets the acceptable result on each dimension. The number of local minima of f_{12} is not known. The global solution and optimum are $x_i^* = i(n+1-i)$ and $-n(n+4)(n-1)/6$. As shown in Table 3, HPSO also gets the best achievements on this function. In short, HPSO is the best to tackle multimodal functions compared with the other methods.

In general, from the analysis in Categories 1 and 2, depending on the global and local mutation operators, HPSO achieves better

balance between global and local searching ability than the other algorithms.

5.2.3. Category 3 – low dimensional functions

To evaluate HPSO more fully, additional benchmark functions are also included in our experiments, i.e., f_{13} – f_{16} , where the number of local minima for each function and the dimension of the functions are small. Table 4 summarizes the results averaged over 50 runs.

Very similar results to the previous ones are obtained despite the large difference in the dimensionality of functions. HPSO still outperforms the other algorithms compared significantly even when the dimension is low. From the results shown in Table 4, by virtue of achieving an effective balance between exploration and exploitation searching abilities, HPSO gets the global optima on all functions except f_{14} . But HPSO also gets the best achievements on f_{14} . This experiment proves that the dimension is not the key factor which determines the behavior of HPSO.

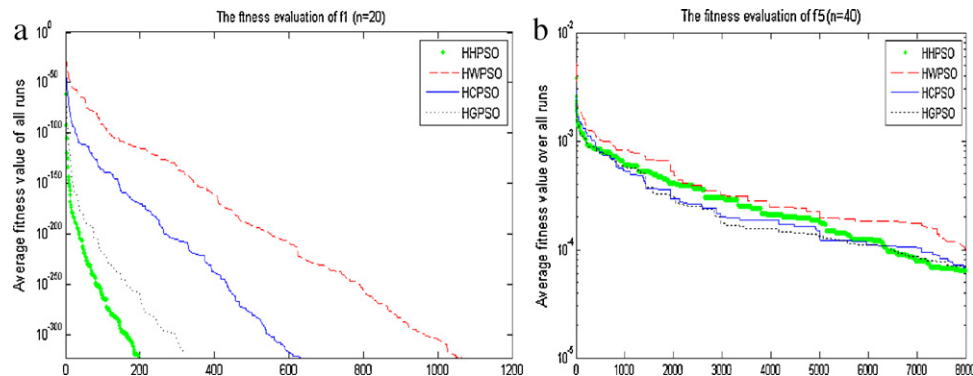


Fig. 8. (a) Function 1 in 20 dimensions. (b) Function 5 in 40 dimensions.

Table 4

Comparison between different PSO methods for benchmark test functions (Category 3).

Function	Dimension	Average (Std. dev.)						
		PSO	CGPSO	HWPSO	HCPSO	IPSO	GPSO	HPSO
f_{13}	2	0.0029664 (0.004427)	0.0034536 (0.003682)	0.00078875 (0.002692)	0.0054374 (0.0048302)	0.0055625 (0.004682)	0.0011793 (0.003807)	0 (0)
f_{14}	2	8.8353e−19 (1.81e−18)	1.2153e−21 (1.149e−21)	1.8406e−25 (3.0479e−25)	6.1405e−21 (4.55e−21)	1.0697e−21 (2.0857e−21)	8.371e−19 (1.29e−19)	4.54e−70 (3.09e−69)
f_{15}	4	2.0868e−13 (2.507e−13)	2.325e−10 (3e−10)	1.207e−13 (4.135e−13)	1.239e−15 (1.39e−15)	2.057e−15 (1.507e−15)	9.205e−14 (9.46e−14)	0 (0)
f_{16}	4	1.396e−72 (9.65e−72)	3.735e−7 (5.78e−7)	1.817e−97 (1.21e−96)	1.935e−83 (5.35e−83)	7.68e−83 (3.46e−82)	3.34e−74 (1.75e−73)	0 0

Table 5

Comparison between different PSO methods for benchmark test functions (Category 4).

Function	Dimension	Average (Std. dev.)						
		PSO	CGPSO	HWPSO	HCPSO	IPSO	GPSO	HPSO
f_1	20	0.057423 (0.10334)	1.243e−15 (4.451e−15)	0.11329 (0.13926)	0.074538 (0.1269)	0.0026465 (0.0055672)	0.073386 (0.13369)	0 (0)
	40	27.569 (30.001)	0.27669 (0.39621)	34.432 (30.327)	14.939 (23.86)	8.7529 (12.072)	45.3 (36.415)	0 (0)
f_5	20	0.018664 (0.011124)	0.028116 (0.018793)	0.018741 (0.013032)	0.016741 (0.0082902)	0.0051485 (0.0031299)	0.012479 (0.009333)	2.8625e−4 3.3911e−4
	40	0.075603 (0.041096)	0.16438 (0.096316)	0.091662 (0.044428)	0.053733 (0.0206)	0.011744 (0.0066711)	0.047077 (0.024543)	8.2608e−5 (7.8585e−5)
f_7	20	36.169 (12.581)	33.704 (7.9642)	25.734 (9.9575)	25.033 (9.9861)	27.904 (7.6383)	25.669 (7.8524)	0 (0)
	40	87.955 (18.497)	91.316 (19.501)	79.188 (22.336)	80.399 (18.149)	67.954 (15.001)	71.488 (20.407)	0 (0)
f_8	20	4.1083 (1.2257)	4.2244 (1.6834)	3.5373 (1.4088)	4.1851 (1.5095)	4.2662 (1.3426)	4.8382 (1.7745)	1.2213e−15 (3.583e−15)
	40	5.8725 (1.4331)	6.4517 (1.8092)	5.3752 (1.6104)	4.9871 (1.8403)	5.5688 (1.4107)	4.7334 (1.9438)	3.565e−15 (4.983e−15)

Table 6

Comparison between PSO with different global mutation operators for Functions 7 and 9.

Function	Dimension	Average (Std. dev.)			
		HGPSO	HWPSO	HGPSO	HGPSO
f_7	20	2.901e–4 (2.1468e–4)	4.2729e–4 (3.7383e–4)	4.2601e–4 (4.9399e–4)	1.1947e–4 (9.6236e–5)
	40	0.11 (0.3001)	0.1623 (0.4243)	0.0758 (0.2495)	0.0033 (0.0025)
f_9	20	2.3236e–7 (2.3173e–7)	1.9235e–7 (2.0924e–7)	1.772e–7 (1.4858e–7)	6.8281e–8 (8.7725e–8)
	40	6.9017e–6 (4.6713e–6)	5.6692e–6 (3.6084e–6)	6.7625e–6 (5.9163e–6)	1.1747e–6 (8.5879e–7)

5.2.4. Category 4 – composition functions

The test functions in Table 1 have been introduced into the literature because of their interesting property, but some of them (e.g. f_1, f_5) lack nonlinear interactions between variables. Further to reveal the search ability of our algorithm, we also test the f_1, f_5, f_7 , and f_8 on 20 and 40 dimensions with more challenging composition. This has the effect of making the problem harder at higher dimensions and also makes the function more resistant to hill-climbing (local mutation method).

Simple function composition can be used to convert these functions into a nonlinear function of 2 variables [31,32]. A separable function of the form

$$F(x_1, x_2, \dots, x_n) = \sum_{i=1}^n \text{Sub}(x_i) \quad (15)$$

Becomes

$$E - F(x_1, x_2, \dots, x_n) = \text{Sub}(\text{Trans}(x_n, x_1)) + \sum_{i=1}^{n-1} \text{Sub}(\text{Trans}(x_i, x_{i+1})) \quad (16)$$

$$\text{Trans}(x, y) = \frac{x + y}{2}$$

where *Sub* is the sub-function used in the original function and the transformation function *Trans* maps two variables onto the domain of *Sub*. This technique provides a method for constructing complex nonlinear fitness landscapes.

For unimodal functions, the role of composition function is to investigate the local search ability of each algorithm. From the results shown in Fig. 7, although the composite function weakens the ability of local mutation operator, HPSO still gets the best results among the algorithms compared. We also test f_2 – f_4 and f_6 on each dimension and get similar results as with f_1 . It is due to the fact that the local mutation operator helps the particle in HPSO to make effective local searching. Furthermore, the composite function also has the effect of making the multimodal functions harder to find the global optimum. By virtue of a higher probability of escaping from a local optimum, HPSO gets more stable and precise results than the others. From the results of Table 5 and Fig. 7, we can con-

clude that HPSO gets the better balance between global and local searching abilities even on composition functions.

5.3. Choosing the global and local mutation operators

To show which function described in Section 4, Part 2 is suitable for using as global or local mutation operator, we test PSO algorithm with different local or global mutation operators for the unimodal functions f_1 and f_5 , the multimodal functions f_7 and f_9 on dimensions 20 and 40.

As described in Section 4, Part 2, the main role of the local mutation operator is to make precise search in the local area of *gbest*, so we only test PSO with Gaussian local mutation operator (HGPSO), PSO with Cauchy local mutation operator (HGPSO), PSO with Henon local mutation operator (HGPSO) and PSO with Wavelet local mutation operator (HWPSO) on unimodal functions f_1 and f_5 with dimensions 20 and 40. Furthermore, the convergence rates of the algorithm are more interesting than the final results of optimization in the test of unimodal functions. So in this comparison we only show the convergence rate of the tested algorithms. As the results shown in Fig. 8(a), we find each compared algorithm has reached the global optima on f_1 and HGPSO performs better than the other algorithms in terms of convergence rate. We also test f_2 – f_4 and f_6 on each dimension and get the similar results as with f_1 . Furthermore, the four algorithms compared show similar convergence rate on f_5 , but HGPSO also gets the best results among the algorithms compared. It is quite clear that the performance of HGPSO improved much more than that of the others since the Henon map has a weaker fine-tuning ability in small to mid-range regions. Furthermore, the smaller average distance between *gbest* and the global optimum favors a small step size and this is also an essential reason that our scheme possesses a high achievement on the unimodal functions. So we choose the Henon function as the local mutation operator.

Functions f_7 and f_9 are multimodal functions and in this test we mainly focus on which function is suitable for using as the global mutation operator. We test the PSO with the four kinds of global mutation operators on f_7 and f_9 . From the results shown in Table 6, we find that PSO with Henon global mutation gets the best results among the algorithms we compared. As described in Section 4, Part 2 the Henon map has a higher probability of generating an offspring further away from

Table 7Comparison for different fixed values of k in HPSO parameters for benchmark test functions.

Function	Dimension	k				
		10	20	30	50	100
f_1	20	0	0	0	0	0
	40	0	0	0	0	0
f_5	20	1.7693e–4 (1.2638e–4)	1.2291e–4 (1.1264e–4)	1.0556e–4 (1.0902e–4)	9.502e–5 (8.3924e–5)	6.6779e–5 (5.3606e–5)
	40	6.3586e–5 (6.2862e–5)	5.7126e–5 (1.0224e–4)	4.2719e–5 (3.4973e–5)	3.9779e–5 (3.4957e–5)	2.9107e–5 (2.3296e–5)
f_7	20	0	0	0	0	0
	40	0	0	0	0	0
f_9	20	6.0304e–10 (7.9459e–10)	1.6199e–9 (2.5755e–9)	4.0022e–9 (7.9355e–9)	2.2035e–8 (2.689e–8)	4.1799e–7 (5.9755e–7)
	40	6.6299e–9 (5.308e–9)	4.2631e–8 (8.0661e–8)	5.3241e–8 (1.6124e–7)	2.6308e–7 (4.8141e–7)	6.4623e–7 (1.6892e–6)

All results are averaged once over 50 runs.

its parent than the other functions and this is the main reason why HPSO gets the best results on multimodal functions. So we choose the Henon function as the global mutation operator.

5.4. Sensitivity of the parameter for the HPSO

In this experiment, we test k and m for the unimodal functions f_1 and f_5 , the multimodal functions f_7 and f_9 on dimensions 20 and 40. The mean cost value and the standard deviation offered by HPSO with different k and m are tabulated in Tables 7 and 8.

In HPSO, k controls the mutation frequency of particle and we want to get the best k of HPSO. These functions are tested by using $k = 10, 20, 30, 50, 100$ with $m = 10$. As it is shown in Table 7, the results on each k have little difference on unimodal functions. But with $k = 10$ we get more favorable results on multimodal functions. As a result, we often choose $k = 10$.

Since m controls the declining rate of T_d , we test the functions by using $m = 2, 5, 10, 20, 25, 50, 100$. All the results are tabulated in Table 8 and in this experiment k is fixed as 10. As the results shown in Table 8, often, we set $m = 10$ which gets a better balance between global and local searching abilities.

From the results shown in Tables 7 and 8, we get little difference in the results by testing HPSO with different k and m . So we can conclude that the achievement of HPSO is not very sensitive to these parameters.

6. Experimental studies of the optimal algorithms in economic dispatch

The operation planning of a power system is characterized by having to maintain a high degree of economy and reliability. Among the options that are available to the engineer in choosing how to operate the system, the most significant choice is in economic dispatch (ED). The ED problem is how to allocate loads to generators for the minimum cost while meeting the constraints. It is formulated as an optimization problem of minimizing the total fuel cost of all committed generators while meeting the demand and losses.

In this section, we apply HPSO to solve the ED problem. We name it the hybrid particle swarm ED (HPSOED). We present some simulation results and comparisons to demonstrate the advantage of our algorithm. The performance of HPSOED is evaluated via computer simulations and compared with GAED (genetic algorithm ED [33]), PSOED (particle swarm ED [21]), IQPSOED (improved quantum particle swarm ED [34]) and the other hybrid PSO algorithms.

6.1. Problem statements

The basic economic dispatch problem can be described mathematically as a minimization problem of minimizing the total fuel cost of all committed generators subject to the constraints. The input–output characteristics of modern generators are nonlinear by nature because of the valve-point loadings and rate limits. Thus, the characteristics of ED problems are multimodal, discontinuous, and highly nonlinear.

$$\text{Minimize } \sum_{i=1}^n F_i(P_i) \quad (17)$$

$F_i(P_i)$ is the fuel cost equation of the i th generator. It is the variation of fuel cost with generated power (MW). Normally it is expressed as continuous quadratic equation.

$$F_i(P_i) = a_i P_i^2 + b_i P_i + c_i, \quad P_i^{\min} \leq P_i \leq P_i^{\max} \quad (18)$$

Table 8
Comparison for different fixed values of m in HPSO parameters for benchmark test functions.

Function	Dimension	m	2	5	10	20	30	50	100
f_1	20	0	0	0	0	0	0	0	0
	40	0	0	0	0	0	0	0	0
f_5	20	9.3692e-5 (9.27e-5)	1.4728e-4 (1.76e-4)	1.7693e-4 (1.2638e-4)	1.7324e-4 (1.92e-4)	2.0088e-4 (2.46e-4)	2.2454e-4 (2.47e-4)	2.1993e-4 (1.86e-4)	
	40	5.3367e-5 (5.68e-5)	6.1623e-5 (5.59e-5)	6.3586e-5 (6.2862e-5)	8.618e-5 (8.24e-5)	8.0287e-5 (9.32e-5)	9.2233e-5 (8.77e-5)	9.2511e-5 (7.59e-5)	
f_7	20	0	0	0	0	0	0	0	0
	40	0	0	0	0	0	0	0	0
f_9	20	1.3683e-8 (2.142e-8)	1.0237e-9 (1.86e-9)	6.0304e-10 (7.9459e-10)	3.0963e-10 (3.803e-10)	2.3151e-10 (2.99e-10)	2.0447e-10 (3.09e-10)	8.2144e-11 (9.23e-11)	
	40	7.91e-8 (2.27e-7)	1.687e-8 (2.84e-8)	6.6299e-9 (5.308e-9)	5.2122e-9 (6.69e-9)	4.4908e-9 (4.64e-9)	2.3443e-9 (2.39e-9)	2.2932e-9 (4.23e-9)	

All results are averaged once over 50 runs

Table 9

Comparison between different algorithms for a 5-Generator system (Rank: 1 – Best, 9 – Worst).

	GAED	IQPSOED	PSO	PSOED	GPSO	HPSO	IPSO	CPSO	HPSOED
Mean	52693.2	8546.572	35664.84	116250.3	46113.02	29200.42	20322.75	12925.76	8409.75
Std	52933.21	218.2795	51705.141	36652.396	39243.2982	38432.291	33891.305	14179.73	47.739
Rank	8	2	6	9	7	5	4	3	1

The total generation should meet the total demand and transmission loss. The transmission loss can be determined from either B_{ij} coefficients or power flow.

$$\sum_{i=1}^n P_i = G + P_l \quad (19)$$

$$P_l = \sum_{i=1}^n \sum_{j=1}^n B_{ij} P_i P_j \quad (20)$$

where G is the load demand, P_i is the output power of the i th generator, P_l is the transmission loss.

6.2. Solution by HPSO

In this section, HPSO is used to solve the ED problem. The particle (solution representation) is defined as follows:

$$p = [P_1, P_2, \dots, P_n] \quad (21)$$

where n denotes the number of generators.

Convert the constrained optimization problems as an unconstrained problem by penalty function method.

$$\text{Minimize } \sum_{i=1}^n F_i(P_i) + 1000 * \text{abs} \left(\sum_{i=1}^n P_i - G - \sum_{i=1}^n \sum_{j=1}^n B_{ij} P_i P_j \right) \quad (22)$$

The stopping condition as the maximum number of iterations is 20,000 or the number of iterations in which the difference between $f(P_g(t))$ and $f(P_g(t+1))$ is less than 0.01 reaches 200.

6.3. Case study

In this part, different hybrid PSO and GA methods are applied to 5-Generator system. This system is taken from [35]. The basic settings of the parameters in the PSOs are the same as those in Section 5, Part 1. All the simulation results are averaged once after 50 runs. The statistical results of the 5-Generator problem in terms of the mean cost value, the standard deviation, and the ranking are shown in Table 9.

By these experiments, we observe that HPSOED shows more precise and stable results than the others. It is mainly attributed to the effective global mutation operator which enables the particle in HPSO to have more chance to roam in the global area. This ability guarantees that the HPSOED gets the best approximate solution among the algorithms we tested. Furthermore, the local mutation operator also helps HPSOED to get more precise results. In general, HPSO is successfully validated for ED system with 5 generators.

7. Conclusion

In this paper, we have described a novel particle swarm optimization with hybrid mutation strategy. Our objective is to apply the new method to find a good balance between global and local searching abilities of PSO. In our proposed algorithm, we use a global mutation operator to enhance the global searching ability of the HPSO. We also used the local mutation operator to make more

precise searching. As a result, the proposed algorithm has the automatic balance ability between exploration and exploitation. Finally, we applied HPSO to solve the ED problem. It greatly improved the generator-system capacity in acceptable computational cost for practical implementation in power systems. The application of HPSO to other problems deserves our further research.

Acknowledgments

This work is supported by the National Basic Research Program of China (973 Program, no. 2010CB731800), the grants from Distinguished Young Scholars of NSFC (no. 6052511) and Innovative Group Grant of NSFC (no. 60721003). The authors of the paper express great acknowledgment for these supports.

References

- [1] J. Kennedy, R.C. Eberhart, Particle swarm optimization, Proc. IEEE Int. Conf. Neural Networks 4 (1995) 1942–1948.
- [2] J. Kennedy, R.C. Eberhart, Swarm Intelligence, Morgan Kaufmann, San Mateo, CA, 2001.
- [3] V.S. Pappala, I. Erlich, K. Rohrig, J.A. Dobschinski, A stochastic modal for the optimal operation of a wind-thermal power system, IEEE Trans. Power Syst. 24 (May (2)) (2009) 940–950.
- [4] P. Acharyee, S.K. Goswami, A decoupled power flow algorithm using particle swarm optimization technique, Energy Convers. Manage. 50 (September (9)) (2009) 2351–2360.
- [5] S. Dutta, S.P. Singh, Optimal rescheduling of generators for congestion management based on particle swarm optimization, IEEE Trans. Power Syst. 23 (November (4)) (2008) 1560.
- [6] S. Kiranyaz, T. Ince, A. Yildirim, M. Gabbouj, Evolutionary artificial neural networks by multi-dimensional particle swarm optimization, Neural Networks 22 (December (10)) (2009) 1448–1462.
- [7] H.L. Wei, S.A. Billings, Y.F. Zhao, L.Z. Guo, Lattice dynamical wavelet neural networks implemented using particle swarm optimization for spatio-temporal system identification, IEEE Trans. Neural Network 20 (January (1)) (2009) 181–185.
- [8] C.J. Lin, C.H. Chen, C.T. Lin, A hybrid of cooperative particle swarm optimization and cultural algorithm for neural fuzzy networks and its prediction applications, IEEE Trans. Syst. Man Cybernetics Part C 39 (January (1)) (2009) 55–68.
- [9] L. Zhao, F. Qian, Y. Yang, Y. Zeng, H. Su, Automatically extracting T-S fuzzy models using cooperative random learning particle swarm optimization, Appl. Soft Comput. 10 (June (3)) (2010) 938–944.
- [10] B. Jiao, Z.G. Lian, X.S. Gu, A dynamic inertia weight particle swarm optimization algorithm, Chaos Solitons Fractal (September) (2008) 698–705, 1362–1381.
- [11] H. Gao, Wenbo Xu, A new particle swarm algorithm and its globally convergent modifications, IEEE Trans. Syst., Man, and Cybern. B, Cybern., in press.
- [12] S.H. Ling, H.H.C. Lu, K.Y. Chan, et al., Hybrid particle swarm optimization with wavelet mutation and its industrial applications, IEEE Trans. Syst. Man Cybernetics 38 (June (3)) (2008) 743–763.
- [13] X.C. Zhao, A perturbed particle swarm algorithm for numerical optimization, Appl. Soft Comput. 10 (1) (2010) 119–124.
- [14] M.G.H. Omran, A.P. Engelbrecht, A. Salman, Differential evolution based particle swarm optimization, in: IEEE Conf. Swarm Intelligence Symposium (SIS), 2007, pp. 112–119.
- [15] M.R. Chen, X. Li, X. Zhang, Y.Z. Lu, A novel particle swarm optimizer hybridized with extremal optimization, Appl. Soft Comput. 10 (2) (2010) 367–373.
- [16] D. Dumitrescu, B. Lazzerini, L.C. Jain, A. Dumitrescu, Evolutionary Computation, CRC Press, 2000.
- [17] N. Hiquishi, H. Iba, Particle swarm optimization with gaussian mutation, in: IEEE Conf. Swarm Intelligence Symposium (SIS), April 2003, pp. 72–79.
- [18] P.S. Andrews, An investigation into mutation operators for particle swarm optimization, in: IEEE Cong. on Evol. Comput., September 2006, pp. 1044–1051.
- [19] H. Wang, Y. Liu, A hybrid particle swarm algorithm with cauchy mutation, in: IEEE Conf. Swarm Intelligence Symposium (SIS), 2007, pp. 356–360.
- [20] H. Wang, Y. Liu, Z. Wu, H. Sun, S.Y. Zeng, L.S. Kang, An improved particle swarm optimization with adaptive jumps, in: IEEE Cong. on Evol. Comput., 2008, pp. 392–397.

- [21] S.H. Ling, H.H.C. Iu, K.Y. Chan, H.K. Lam, B.C.W. Yeung, F.H. Leung, Hybrid particle swarm optimization with wavelet mutation and its industrial applications, *IEEE Trans. Syst. Man Cybernetics* 38 (June (3)) (2008) 743–763.
- [22] C. Leandro dos Santos, An efficient particle swarm approach for mixed-integer programming in reliability-redundancy optimization applications, *Reliab. Eng. Syst. Saf.* 94 (April (4)) (2009) 830–837.
- [23] M. Clerc, J. Kennedy, The particle swarm-explosion, stability, and convergence in a multi-dimensional complex space, *IEEE Trans. Evol. Comput.* 6 (1) (2002) 58–73.
- [24] F. van Den Bergh, An analysis of particle swarm optimizers, Ph.D. Dissertation, University of Pretoria, November 2001, unpublished.
- [25] P.P. Boyle, A Monte Carlo approach, *J. Financ. Econ.* 4 (May (3)) (1977) 323–338.
- [26] Y. Shi, R. Eberhart, A modified particle swarm optimizer, *IEEE Int. Conf. Evol. Comput.* (1998) 69–73.
- [27] I. Daubechies, *Ten Lectures on Wavelet*, SIAM, Philadelphia, PA, 1992.
- [28] X. Yao, Y. Liu, Evolutionary programming made faster, *IEEE Trans. Evol. Comput.* 3 (July (2)) (1999) 82–102.
- [29] M.M. Ali, C. Khompatraporn, Z.B. Zabinsky, A numerical evaluation of several stochastic algorithms on selected continuous global optimization test problems, *J. Global Optim.* (November) (2005) 635–672.
- [30] F. Van Den Bergh, A.P. Engelbrecht, Effect of swarm size on cooperative particle swarm optimizers, in: *Proc. Genetic Evolution. Comp. Conf. (GECCO-2001)*, San Francisco, CA, July, 2001, pp. 892–899.
- [31] D. Whitley, R. Beveridge, C. Graves, K. Mathias, Test driving three 1995 genetic algorithms: new test functions and geometric matching, *J. Heuristics* 1 (September (1)) (1995) 77–104.
- [32] D. Whitley, K. Mathias, S. Rana, J. Dzubera, Evaluating evolutionary algorithms, *Artif. Intell.* 85 (1–2) (1996) 245–276.
- [33] P.H. Chen, H.C. Chang, Large-scale economic dispatch by genetic algorithm, *IEEE Trans. Power Syst.* 10 (February (1)) (1995) 117–124.
- [34] L. dos, S. Coelho, V.C. Mariani, Particle swarm approach based on quantum mechanics and harmonic oscillator potential well for economic load dispatch with valve-point effects, *Energy Convers. Manage.* 49 (11) (2008) 3080–3085.
- [35] H. Saadat, *Power System Analysis*, McGraw-Hill, New York, 1999.